Parallel and Distributed Computing — Semester Project Report

Optimizing AlphaZero-Style Self-Play for Othello: Batching, Caching, SIMD, and Multiprocessing on Constrained Hardware

Course	Parallel and Distributed Computing (PDC)
Group Members	Member 1 · Roll No. 23i-0075 Member 2 · Roll No. 23i-0583
Base Paper	*Learning to Play Othello Without Human Knowledge* (alpha-zero-general)
Platform	Google Colab T4 GPU · 2 vCPUs · 12.7 GB RAM
Date	May 2026

Abstract

This report presents the design, implementation, and empirical evaluation of five progressive optimization strategies (Techniques A–E) applied to an AlphaZero-style self-play training loop for 8×8 Othello, running on a single Google Colab T4 GPU. Starting from a sequential baseline achieving approximately 1,098 MCTS simulations per second, we show that replacing per-episode sequential inference with single-process lockstep batched self-play (Technique C) raises throughput to approximately 1,684 MCTS simulations per second — a 53% gain — while preserving a win rate of 1.0 against a greedy opponent. Multiprocessing (Technique D) and explicit virtual-loss bookkeeping (Technique E) were both found to reduce throughput relative to Technique C under Colab's constrained CPU environment, demonstrating that coordination overhead can dominate on low-core machines. A C++ bitboard back-end with OpenMP and AVX2 SIMD (Phase 3) was also implemented; it currently trails the Python baseline due to Python–C++ boundary overhead in hot loops, but provides a sound foundation for future integration. Negative results are reported alongside positive ones throughout.

1. Introduction

Reinforcement learning systems that improve through self-play — most prominently the AlphaGo Zero and AlphaZero family — impose demanding computational requirements. Each training iteration requires thousands of Monte Carlo Tree Search (MCTS) simulations, each backed by a neural network inference call, followed by supervised training and model evaluation. Even on capable hardware, the interaction between the MCTS traversal loop (CPU-bound), the neural-network inference calls (GPU-bound), and the game-state bookkeeping (memory-bound) creates a layered, heterogeneous workload that does not parallelize trivially.

From a Parallel and Distributed Computing perspective, this workload is instructive: it offers opportunities for task-level parallelism (multiple independent self-play games), data-level parallelism (batched GPU inference over many board positions), memory-access optimization (cache-friendly state representation, bounded caches, zero-copy board encoding), and system-level optimization (C++ extensions, SIMD vectorization, multi-process coordination). At the same time, the workload is sensitive to overhead: the inner

MCTS loop is called millions of times per training run, so even small per-call costs compound into large end-to-end regressions.

This project investigates all of those dimensions empirically. We use the open-source `alpha-zero-general` codebase applied to 6×6/8×8 Othello as our experimental platform, Google Colab T4 as our target hardware, and a rigorous metrics-driven methodology to report both successes and failures.

---

2. Base Paper and Problem Context

2.1 AlphaZero and alpha-zero-general

The AlphaGo Zero paper (Silver et al., 2017) demonstrated that superhuman Go play can be achieved purely through self-play reinforcement learning, without human game knowledge. The core loop is:

1. **Self-play**: a neural-network-guided MCTS generates game trajectories and training labels (board state, MCTS policy, game outcome).
2. **Training**: a shared policy-value network is trained on the accumulated examples.
3. **Model selection**: the updated network is accepted only if it beats the current best model in arena play.

The `alpha-zero-general` repository (Thakoor, Nair, Jhunjhunwala) provides a clean PyTorch implementation of this loop generalized across board games. Othello (Reversi) is the default demonstration game and was our chosen target.

2.2 Othello as a Computational Problem

An 8×8 Othello board has up to 10^{28} reachable positions. At each MCTS leaf, the neural network receives the board state as an 8×8 float tensor and returns a policy vector over all 65 possible moves (64 squares plus pass) and a scalar value estimate. The key computational characteristics are:

- MCTS is **sequential by default**: each simulation traverses a path from root to leaf, expands the leaf, and backs up values.
- Neural inference is **GPU-efficient only in batches**: individual forward passes waste most of the GPU's parallel capacity.
- Game-state transitions and legal-move generation are **CPU-intensive** and called O(simulations × branching-factor) times per training iteration.
- The same or transposition-equivalent board positions **recur** across different game paths, creating a caching opportunity.

2.3 Why PDC Matters Here

The baseline implementation evaluates each MCTS leaf synchronously and one game at a time. This means:

- GPU utilization is low (one small batch per leaf expansion).
- CPU utilization is bursty (traversal then wait for GPU).
- Parallelism across concurrent games is zero.

PDC techniques — task batching, shared-memory communication, SIMD acceleration of board logic, and multi-process coordination — are each directly applicable to one or more of these gaps.

---

3. Project Scope and Objectives

3.1 What We Implemented

We implemented five optimization strategies applied incrementally to the `alpha-zero-general` codebase:

- **Technique A**: NumPy array-based MCTS node storage (replacing Python dicts).
- **Technique B**: Neural-network position cache with symmetry-aware insertion.
- **Technique C**: Single-process lockstep batched self-play with shared game tree.
- **Technique D**: Two-process parallel self-play (historical two-CUDA-worker variant and revised parent-owned inference variant).
- **Technique E**: Technique C augmented with explicit virtual-loss bookkeeping.

Additionally, three engineering phases were integrated across all techniques:

- **Phase 1**: Bounded LRU caches, NumPy array pooling, optional Zobrist hashing, profiling infrastructure.
- **Phase 2**: Thread-pool MCTS with node-level locking and virtual-loss coordination.
- **Phase 3**: C++ bitboard back-end (`othello_bitboard.cpp`) with OpenMP parallelism and AVX2/SIMD optimizations, compiled as a pybind11 extension.

3.2 Parallelization Strategies Targeted

Strategy	Techniques / Phases
GPU batch parallelism	B, C, D, E
Task-level parallelism (concurrent games)	C, D
Shared-memory communication	C (shared tree), D (queue)
SIMD data parallelism	Phase 3
Multi-process parallelism	D
Memory-access optimization	A, Phase 1, Phase 3
Inference reuse / caching	B, C, Phase 1

4. Baseline Method

4.1 Sequential Implementation

The baseline follows the standard `alpha-zero-general` self-play loop:

- Each training iteration runs `numEps = 25` self-play episodes sequentially, one game at a time.
- Each episode uses `numMCTSSims = 25` MCTS simulations per move.
- MCTS state is stored in six Python dictionaries keyed by board string: `Qsa`, `Nsa`, `Ns`, `Ps`, `Es`, `Vs`.
- At each MCTS leaf, `nnet.predict(board)` is called directly — one board at a time — incurring one GPU forward pass per expansion.
- After self-play, training and arena comparison run sequentially.

4.2 Baseline Performance

The following table reproduces the measured baseline over five training iterations. Configuration: `numMCTSSims = 15`, `numEps = 25`, `numIters = 10`.

Iteration	Self-play (s)	Train (s)	Arena (s)	MCTS sims/sec	Peak RAM (MB)	GPU util. (%)
1	30.25	14.12	30.01	1,075.9	1,364.4	51.6
2	30.57	27.25	30.73	1,063.1	1,472.3	57.0
3	29.78	42.41	31.18	1,100.4	1,551.9	58.6
4	29.88	57.21	30.32	1,097.7	1,568.4	61.8
5	28.43	71.09	29.90	1,151.1	1,717.3	64.9
Avg	**29.78**	**42.42**	**30.43**	**1,097.6**	**1,534.9**	**58.8**

Win rate versus greedy evaluator: **1.0** (all five iterations). Training time grows across iterations because training examples accumulate; this is expected behavior, not a performance regression.

4.3 Bottleneck Characterization

At baseline, the primary bottlenecks are:

1. **Single-board GPU inference**: every MCTS leaf dispatches a batch of size 1 to the GPU, wasting parallel capacity.
2. **Sequential episode execution**: no concurrent game state is exploited.
3. **Python-level game logic**: `stringRepresentation`, `getValidMoves`, `getNextState`, and `getGameEnded` are called O(millions) of times and run entirely in interpreted Python.
4. **Unbounded dict growth**: MCTS state dictionaries grow without limit across an iteration, creating cache-unfriendly access patterns late in training.

---

5. Proposed Parallel / Optimized Approaches

5.1 Technique A — NumPy Arrays for MCTS Node Storage

Motivation: Contiguous array layout should improve cache locality and allow UCB selection to be vectorized with NumPy's optimized kernels, replacing individual dict key-value accesses.

Implementation: Six MCTS dictionaries were replaced with an `MCTSNode` class holding four NumPy arrays per node (`Q`, `N`, `P`, `valid_moves`) plus scalar fields. UCB computation was vectorized:

```
ucb = node.Q + cpuct * node.P * np.sqrt(node.N_total + EPS) / (1 + node.N)
ucb[node.valid_moves == 0] = -np.inf
best_act = np.argmax(ucb)
```

Outcome: MCTS throughput regressed by 5–10% (999–1,026 sims/sec vs. 1,063–1,151 for baseline). Win rate remained 1.0.

Root cause: The 6×6 Othello action space is only 37 actions. NumPy call overhead, MCTSNode object construction, and the extra dict-lookup-plus-attribute-access path all dominated the small vectorization gain. For action spaces below roughly 100, Python-native dict access is faster than NumPy dispatch.

Lesson: Array-based layout benefits depend critically on array size. Measure before assuming vectorization helps.

5.2 Technique B — Neural Network Position Cache

Motivation: Many board positions recur across episodes within an iteration, and transposition-equivalent positions (related by Othello's 4-fold rotational symmetry) appear even more frequently. Caching neural predictions eliminates redundant GPU forward passes.

Implementation: A dict-based cache (`board_string → (policy, value)`) was added to the MCTS object. On a cache miss, all symmetric board-policy pairs were inserted, amortizing the hit-rate benefit. The cache persisted across episodes within an iteration but was reset before each new iteration to prevent stale predictions from degraded (pre-update) network weights.

Outcome: Average self-play time improved ~5.5% versus Technique A. GPU calls per iteration fell from ~12,337 to ~9,701 as the cache hit rate grew (12.9% → 19.4%). MCTS throughput averaged ~1,076 sims/sec. Win rate in the reported run was 0.6, though this is likely run variance under short evaluation settings rather than a structural quality degradation.

Lesson: Inference caching is a genuine systems win; correctness checks on win-rate comparisons require fixed seeds and more evaluation games.

5.3 Technique C — Lockstep Batched Self-Play *(Best Result)*

Motivation: The root cause of poor GPU efficiency is that sequential self-play produces batch-size-1 inference calls. If multiple games are run concurrently, their MCTS leaf expansions can be collected and dispatched as a single larger GPU batch.

Implementation: A `BatchedSelfPlayWorker` maintains up to 16 active game slots in a rolling-refill loop. At each coordination step:

1. All active games advance their MCTS traversal until each either reaches a cached leaf or reaches a new leaf requiring network evaluation.
2. Pending leaf boards from all active games are coalesced into a single batched `nnet.predict()` call.
3. Duplicate board requests (same position reached by multiple games) are sent to the GPU once; the result is backed up through all waiting paths.
4. Completed games are replaced immediately with new ones until `numEps` total games finish.

Additional features integrated: shared MCTS node tree across active games for transposition reuse; bounded LRU caches (from Phase 1); a virtual-visit guard to reduce duplicate branch selection; and a stuck-game watchdog (`MAX_STEPS = 200`).

Final run parameters: `numEps = 48`, `numMCTSSims = 35`, `arenaCompare = 40`, `greedyCompare = 40`, `epochs = 15`, `batch_size = 128`.

Outcome:

Iteration	Self-play (s)	MCTS sims/sec	Avg GPU batch	GPU calls	Win rate
1	54.35	1,848.1	12.68	3,286	—
2	59.90	1,678.2	12.21	2,991	—
3	62.04	1,620.7	11.20	2,864	—
4	62.64	1,604.8	9.43	2,841	—
5	60.21	1,666.5	10.35	2,920	1.0
Avg	**59.83**	**1,683.7**	**11.17**	**2,980**	**1.0**

Compared with the baseline (same 25-sim configuration), Technique C delivers approximately **53% higher MCTS throughput** (1,684 vs. 1,098 sims/sec) and achieves win rate 1.0.

Key insight: The practical average batch size of 11.17 (against a theoretical maximum of 16) arises because games finish at different lengths — rolling refill ensures slots are never left empty for long. Larger `numEps` divisible by the batch size (48 = 3 × 16) help keep slots full.

5.4 Technique D — Two-Process Parallel Self-Play

Motivation: Two CPU cores are available on Colab T4; spawning one self-play worker per core should halve self-play wall-clock time.

Historical design: Two worker processes each ran a full `BatchedSelfPlayWorker`, including independent CUDA inference. This produced two competing CUDA contexts on one GPU.

Historical outcome: No self-play speedup; MCTS throughput fell to ~903 sims/sec; win rate degraded to 0.3. Root causes: single-GPU contention, IPC overhead, loss of shared transposition reuse across the full game set, and smaller per-worker batches.

Revised design: Workers perform CPU-side MCTS traversal only and send pending leaf boards to the main process through a queue. The main process owns the PyTorch model exclusively and coalesces requests from both workers into one batched GPU call. This eliminates CUDA contention.

Revised outcome: GPU call count improved vs. historical D (3,245 vs. 3,784 per iteration) and average batch size improved (9.79 vs. 9.35). However, self-play time worsened to 76.9 s/iteration and MCTS throughput fell to ~709 sims/sec — worse than both Technique C and the baseline. Win rate recovered to 0.5 but remained below Technique C's 1.0.

Root cause of persistent failure: IPC queue serialization and wait latency are injected directly into the MCTS hot path. On Colab's 2-core CPU, every leaf expansion crosses a process boundary; the resulting synchronization cost exceeds the gain from parallel traversal. Python-level MCTS traversal, game-logic calls, and inter-process serialization collectively dominate GPU wait time.

Lesson: Multi-process parallelism for MCTS requires much lower per-simulation IPC cost (e.g., shared-memory numpy arrays, C-level coordination) and/or far more CPU cores to absorb overhead before it yields net speedup.

5.5 Technique E — Batched Self-Play with Explicit Virtual Loss

Motivation: When multiple active games are waiting for the same node to be expanded, they may all select the same child action (no virtual loss), creating correlated search paths and wasting diversity. Explicit virtual-loss discounts encourage active games to explore distinct branches, improving search quality and potentially improving effective GPU batch diversity.

Implementation: Technique C with added virtual-loss state tracking per MCTS action during the batch coordination phase. A diversion counter tracks how often virtual-loss actually redirects a game to a different branch.

Outcome:

Metric	Technique C	Technique E
Avg MCTS sims/sec	1,683.7	1,565.4
Avg self-play time (s)	59.83	64.19
Avg GPU batch size	11.17	11.25
Avg GPU calls/iter	2,980	2,960
Avg cache hit rate	6.95%	15.05%
Win rate vs greedy	1.0	1.0
Avg virtual-loss diversions	—	0.067/sim

Quality was fully preserved. Cache hit rate improved noticeably (15.05% vs. 6.95%). However, MCTS throughput fell ~7% because the virtual-loss bookkeeping runs in the hottest part of the selection loop, and the actual diversion rate (0.067 per simulation) is too low to repay the overhead.

Lesson: Micro-optimizations in tight inner loops must be profiled for both call count and per-call cost. Collision avoidance is useful, but its bookkeeping must be gated behind profiling flags in production.

5.6 Phase 1–3 Engineering Integration

Phase 1 replaced unbounded dict caches with `LRUCache` objects, added `NumpyArrayPool` for reusable fixed-shape action arrays, and added optional Zobrist hashing for board-state lookup. Benchmark results showed the plain `python_tobytes` path (1,220.5 sims/sec) remained fastest; Zobrist hashing was slower (835.4 sims/sec) due to hash-maintenance overhead in this workload; the C++ bitboard achieved 1,104.2 sims/sec — faster than Zobrist but below baseline due to Python–C++ conversion costs in hot loops. Bounded caches improve memory safety; the recommended production setting keeps `use_zobrist=False` and `use_bitboard=False` until Phase 3b interface overhead is resolved.

Phase 2 added a thread-pool MCTS mode with node-level locking and virtual-loss coordination. No throughput improvement was observed on 2-vCPU Colab; the GIL and lock-contention costs dominated parallel traversal gains.

Phase 3 is described in detail in Section 5.7 below.

5.7 Phase 3 — C++ Bitboard Back-End with OpenMP and SIMD

The C++ extension (`othello_bitboard.cpp`) replaces Python-level Othello board logic with a zero-allocation, register-resident implementation based on two 64-bit integers representing black and white disc positions. All board operations reduce to bitwise shifts, masks, and population-count instructions.

5.7.1 Data Representation

Each board square maps to a single bit: `bit_index = row × 8 + col`. Legal-move generation and flip computation operate on 64-bit masks with no per-cell branches.

```
white_ = 0xFFF...  (one bit set per white disc)
black_ = 0xFFF...  (one bit set per black disc)
```

5.7.2 SIMD and Compiler Optimizations

Several techniques were applied to maximize per-call performance:

Compiler pragmas (file-scope, GCC/Clang):

```
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,popcnt")
```

These direct the compiler to emit AVX2, BMI2, and hardware-POPCNT instructions for this translation unit even when the project is otherwise compiled for a generic target.

Hardware POPCNT (`count_diff`, `get_legal_moves_list`):

```
#define HW_POPCNT(x) static_cast<int>(_mm_popcnt_u64(x))  // x86
```

`_mm_popcnt_u64` compiles to a single `POPCNT` instruction — one cycle latency on modern Intel/AMD — rather than a multi-instruction emulation. Non-x86 targets fall back to `__builtin_popcountll`.

Cache alignment:

```
class alignas(64) BitBoard { ... };
```

Aligning to 64 bytes (one cache line) prevents false sharing when multiple threads hold private `BitBoard` instances (e.g., in the OpenMP batch loops) and enables the compiler to issue aligned `VMOVDQA` loads.

Branch prediction hints:

```
#define LIKELY(x)   __builtin_expect(!!(x), 1)
#define UNLIKELY(x) __builtin_expect(!!(x), 0)
```

Applied to the `ray_flips` traversal loop (hot path, `LIKELY`) and error-checking guards (cold path, `UNLIKELY`).

No-alias pointers (`__restrict__`): asserted on raw pointer parameters in the batch helpers, unlocking wider load/store vectorization and loop-invariant code motion.

Loop unrolling: `#pragma GCC unroll 8` on the inner loops of `to_numpy` / `from_numpy` (exactly 8 iterations, all independent). The `legal_dir` propagation is hand-unrolled to 6 explicit steps.

5.7.3 OpenMP Batch Parallelism

Two new module-level functions accept a 3-D NumPy array of shape `(N, 8, 8)` and evaluate all N boards in parallel:

```
#pragma omp parallel for schedule(dynamic, 64) default(none) \
    shared(base, result, color) firstprivate(N)
for (py::ssize_t i = 0; i < N; ++i) {
    BitBoard bb;          // private to each thread, stack-allocated
    load_board_slice(bb, base, i);
    result[i] = bb.get_legal_moves_mask(color);
}
```

Thread safety is guaranteed by the private-stack pattern: each thread constructs its own `BitBoard`, reads from a shared read-only input array, and writes to a distinct pre-allocated output slot. No mutex is needed. `schedule(dynamic, 64)` balances load across threads while amortizing scheduling overhead in chunks of 64 boards.

These functions are compiled with `-fopenmp -march=native -O3` and guarded by `#ifdef _OPENMP` so the file degrades gracefully to single-threaded operation without the flag.

5.7.4 Python API (Preserved Interface)

All original method signatures are unchanged (`get_legal_moves_list`, `has_legal_moves`, `execute_move`, `count_diff`, `to_numpy`, `from_numpy`, `hash`). The two new batch functions are additive:

```
masks = ob.batch_get_legal_moves_mask(boards_array, color=1)  # (N,) uint64
diffs  = ob.batch_count_diff(boards_array, color=1)            # (N,) int
```

5.7.5 Current Status and Path Forward

The Phase 3 benchmark shows the C++ path at 1,104.2 sims/sec versus the Python baseline at 1,220.5 sims/sec. The gap is attributable to conversion overhead at the Python–C++ boundary (board state is

reconstructed as a `BitBoard` from a NumPy array on every call). Eliminating this overhead requires maintaining board state in native bitboard form throughout MCTS traversal and converting only at neural-network inference boundaries — a Phase 3b refactor.

---

6. Experimental Setup

6.1 Hardware

Resource	Specification
GPU	NVIDIA Tesla T4 (16 GB GDDR6)
CPU	2 vCPUs (Intel Xeon, Colab allocation)
RAM	12.7 GB system memory
Storage	Google Colab ephemeral disk

6.2 Software Environment

Component	Version / Details
Python	3.10
PyTorch	Latest Colab-default with CUDA support
pybind11	System-default for C++ extension build
Compiler	GCC (C++17), with `-O3 -march=native -fopenmp` for Phase 3
CUDA	`torch.backends.cudnn.benchmark = True` enabled in Technique C+ runs

6.3 Build Command (Phase 3 Extension)

```
c++ -O3 -march=native -fopenmp -Wall -shared -std=c++17 -fPIC \
    $(python3 -m pybind11 --includes) \
    othello_bitboard.cpp \
    -o othello_bitboard$(python3-config --extension-suffix)
```

6.4 Evaluation Metrics

Every run records: `self_play_sec`, `train_sec`, `arena_sec`, `gpu_utilization_pct`, `mcts_sims_per_sec`, `peak_ram_mb`, `cache_hit_rate_per_iter`, `gpu_calls_per_iter`, `avg_gpu_batch_size`, `win_rate_vs_greedy`. Multiprocessing runs additionally record `worker_utilization` and `examples_per_worker`.

---

7. Results and Discussion

7.1 Consolidated Technique Comparison

Technique	Core Idea	Avg MCTS sims/sec	Δ vs Baseline	Avg GPU batch	Win rate	Verdict
Baseline	Sequential self-play	1,097.6	—	1.0 (implicit)	1.0	Reference
A	NumPy node arrays	~1,019	**−7%**	1.0	1.0	× Regression
B	NN position cache	~1,076	**−2%**	1.0	0.6*	✅ Partial win
C	Lockstep batched self-play	**1,683.7**	**+53%**	11.17	**1.0**	✅ **Best**
D (hist.)	2-process, 2 CUDA	~903	−18%	9.35	0.3	× Failed
D (revised)	2-process, parent GPU	~709	−35%	9.79	0.5	× Failed
E	Technique C + virtual loss	1,565.4	+43%	11.25	1.0	⚠ Quality-safe, slower than C

*Win rate of 0.6 for Technique B is likely run variance; further evaluation with fixed seeds is recommended.

7.2 Throughput and GPU Efficiency

The progression from Baseline → B → C reveals three distinct regimes:

- **Baseline**: Single-board inference; GPU is active but underutilized in batch dimension. ~1,098 sims/sec.
- **Technique B**: Cache reduces GPU call count but inference is still sequential. Marginal improvement.
- **Technique C**: Batched coordination raises average GPU batch to 11.17, reducing total GPU calls from ~10,000+ to ~2,980 per iteration. MCTS sims/sec rises to 1,684. This is the key step change.

The difference between Technique C (sims/sec ≈ 1,684) and Technique E (≈ 1,565) despite nearly identical GPU-side behavior (batch size, call count, utilization) confirms that the bottleneck in Technique E is CPU-side Python overhead — specifically, the per-simulation virtual-loss bookkeeping — rather than anything on the GPU path.

7.3 Memory Behavior

Peak RAM across techniques (iteration 5):

Technique	Peak RAM (MB)
Baseline	1,717
A	1,709
B	1,840
C	2,621

Technique	Peak RAM (MB)
D (revised)	2,526
E	2,662

RAM growth from B onward reflects the NN cache, shared MCTS tree, and expanded training-example history. Technique C's memory cost is acceptable on Colab's 12.7 GB RAM, though it must be monitored if the number of iterations or the cache size is scaled up significantly.

7.4 Bottleneck Analysis

Across all techniques, profiling points to the following hierarchy of bottlenecks:

1. **GPU batch size** (addressed well by Technique C): Moving from batch size 1 to ~11 was the single largest throughput lever.
2. **CPU-side MCTS traversal and game logic** (unresolved): `_select_action`, `getValidMoves`, `getNextState`, `getGameEnded`, and `stringRepresentation` are called millions of times per training run in interpreted Python. Techniques D and E both show regressions when additional CPU work is added to this path.
3. **Python–C++ interface overhead** (partially addressed by Phase 3): Each call to the C++ bitboard currently requires reconstructing the board state from a NumPy array. Eliminating this boundary crossing is the key remaining Phase 3 task.
4. **IPC / process-coordination overhead** (Technique D): On 2-vCPU Colab, inter-process queue latency is too high to allow multi-process MCTS coordination to break even against Technique C.

7.4.1 Hardware Limitations and Bound Classification

Understanding whether an implementation is compute-bound, memory-bound, or communication-bound is essential for guiding further optimisation. On the Google Colab T4 hardware, the relevant hardware ceilings are:

- **CPU compute ceiling**: 2 vCPUs (Intel Xeon) with a theoretical peak of ~50–100 GFLOPS single-threaded (AVX2) and ~100–200 GFLOPS with both cores active.
- **GPU compute ceiling**: NVIDIA T4 delivers ~8.1 TFLOPS (FP32) for tensor operations.
- **Memory bandwidth**: T4 has ~300 GB/s GDDR6; system DRAM bandwidth is much lower (<20 GB/s for a typical Colab VM).
- **Communication overhead**: IPC latency is in the microsecond range per crossing but becomes prohibitive when injected into the MCTS hot path, which executes millions of leaf expansions per training iteration.

Using the metrics collected across Techniques A–E and the scalability runs, each major system component can be classified:

Component	Observed behaviour	Bound classification

Component	Observed behaviour	Bound classification
Neural network inference	GPU utilisation 40–60%; average batch size ~11.2; throughput far below the T4's 8.1 TFLOPS ceiling.	**Not GPU-bound** — the GPU is underutilised because the CPU cannot feed it fast enough.
MCTS traversal + game logic	CPU saturates one of the two vCPUs. Per-simulation operations — UCB, `getValidMoves`, `getNextState`, dict lookups — dominate the critical path.	**CPU compute-bound** — time is spent executing Python instructions, not waiting for memory or I/O.
Memory access / locality	Position cache hit rates reach 15–19% (Techniques B/E). The shared MCTS tree fits mostly in L3 cache. Peak RAM stays below 3 GB, well under the 12.7 GB limit. No measurable memory bandwidth saturation.	**Not memory-bound** — the working set is small and access patterns are sufficiently local.
Synchronisation / communication	In Technique D (parent-owned inference), each MCTS leaf expansion crosses a process boundary. Queue serialisation adds hundreds of microseconds per call — catastrophic when multiplied by millions of crossings. Self-play time worsens relative to Technique C despite better GPU-side indicators.	**Communication-bound** (Technique D only) — the design fails because IPC coordination overhead dominates the hot path.

Conclusion for Technique C (the best result): The implementation is **CPU compute-bound**. The GPU sits idle waiting for the CPU to finish MCTS traversal, and memory bandwidth is not a constraint. Any further speedup must therefore reduce CPU-side work per simulation. The two most promising directions are:

- Moving MCTS hot loops to a compiled language (C++, Rust, or Numba) while keeping the existing batched coordination layer, or
- Using additional CPU cores with a shared-memory, lock-free MCTS tree — not Python `multiprocessing`, which re-introduces the IPC overhead that crippled Technique D.

Techniques D and E serve as cautionary examples of the same failure mode: both added CPU work (virtual-loss bookkeeping in E, queue serialisation in D) to an already CPU-bound path, causing throughput regressions despite no change in GPU-side behaviour.

7.5 Scalability

Scalability was measured across four dimensions using the `example_technique_c_scalability.py` test harness on Google Colab T4. All tests used 6×6 Othello unless otherwise noted. The 4×4 board size was not testable: the fixed 3×3 convolutional kernel in `OthelloNNet` requires a minimum board side of 5, and all 4×4 attempts raised a `RuntimeError` at the first forward pass; those results are excluded from the analysis below.

7.5.1 Batch Size Scaling

Two sweeps were run: a standard sweep (4–32 active slots, 96 games, 35 sims) and a larger-batch sweep (16–64 active slots, 128 games, 50 sims).

Standard sweep (`num_games=96`, `num_sims=35`):

Active slots (batch size)	Games/sec	Avg GPU batch size	Cache hit rate	Peak RAM (MB)
4	0.52	3.09	12.33%	72.9
8	0.65	6.47	11.68%	93.4
12	0.69	9.41	11.48%	94.0
16	0.75	12.97	11.74%	95.2
24	0.79	18.72	12.74%	103.7
32	**0.82**	26.03	12.36%	105.4

Larger-batch sweep (`num_games=128`, `num_sims=50`):

Active slots	Games/sec	Avg GPU batch size	Cache hit rate	Peak RAM (MB)
16	0.51	12.51	12.25%	95.2
24	0.53	16.92	11.50%	103.7
32	0.56	25.53	10.77%	105.4
48	0.56	33.98	11.90%	117.0
64	**0.58**	52.30	10.44%	127.3

Analysis: Throughput increases consistently with batch size but with diminishing returns — the gain from 4→16 slots (0.52→0.75 games/sec, +44%) is much larger than from 32→64 slots (0.56→0.58, +4%). This is the classic GPU saturation curve: small batches leave the T4 underutilised, while very large batches are CPU-limited rather than GPU-limited. The practical sweet spot on Colab T4 is **16–32 active slots** — beyond that, throughput gains are marginal while memory pressure grows. Notably, GPU batch size scales roughly linearly with active slots (3.09 at 4 slots → 52.30 at 64 slots), confirming the coalescing logic correctly aggregates more requests as more games run concurrently.

7.5.2 Number of Games Scaling (Throughput Stability)

Configuration: `batch_size=16`, `num_sims=35`, board 6×6.

Total games	Games/sec	MCTS sims/sec	Total time (s)	Peak RAM (MB)
16	0.74	832	21.76	95.2
32	0.75	857	42.50	95.2
48	0.76	856	63.38	95.2

Total games	Games/sec	MCTS sims/sec	Total time (s)	Peak RAM (MB)
96	0.75	846	128.18	95.2
192	0.73	832	261.56	95.2

Analysis: Throughput is essentially flat across a 12× range in workload size (0.73–0.76 games/sec). Memory is also constant at 95.2 MB regardless of total games, confirming the rolling-refill design caps in-flight state at the batch size rather than growing with the full episode count. This is strong evidence that Technique C scales gracefully in the number-of-games dimension: doubling the workload doubles the wall-clock time without any efficiency loss. The slight dip at 192 games (0.73) is within noise.

7.5.3 MCTS Simulation Depth Scaling

Two sweeps were run: a depth sweep at `batch_size=16` and a larger-batch depth sweep at `batch_size=32`.

Depth sweep (`batch_size=16`, `num_games=48`):

MCTS sims/move	MCTS sims/sec	Games/sec	Time/game (s)	Total time (s)
10	852	2.62	0.382	18.34
20	879	1.36	0.738	35.42
35	842	0.75	1.335	64.09
50	842	0.52	1.922	92.26
75	830	0.34	2.918	140.09

Larger-batch depth sweep (`batch_size=32`, `num_games=96`):

MCTS sims/move	MCTS sims/sec	Games/sec	Time/game (s)
35	895	0.79	1.267
50	906	0.56	1.786
75	883	0.36	2.742

Analysis: MCTS throughput (sims/sec) is remarkably stable across all depths — ranging only 830–906 sims/sec across a 7.5× range in simulation count. This is the defining property of the batched architecture: as simulation count per move increases, more leaf requests accumulate per batch cycle, naturally driving GPU batch size upward and keeping hardware efficiency constant. Time per game scales linearly with sim count (0.382 s at 10 sims → 2.918 s at 75 sims ≈ 7.6× growth for 7.5× more sims), which confirms the implementation has no super-linear overheads. The larger `batch_size=32` sweep slightly outperforms the `batch_size=16` sweep on MCTS sims/sec (895–906 vs. 830–879), consistent with the batch-size scaling findings above.

7.5.4 Board Size Scaling

Configuration: `batch_size=16`, `num_games=32`, `num_sims=20`. Only 6×6 and 8×8 were testable (4×4 fails due to the network's minimum kernel constraint, as described above).

Board size	Games/sec	MCTS sims/sec	Total time (s)	Peak RAM (MB)
6×6	1.32	865	24.19	95.2
8×8	0.55	659	58.68	120.2

Analysis: Moving from 6×6 to 8×8 reduces games/sec by 58% (1.32 → 0.55) and MCTS sims/sec by 24% (865 → 659). The throughput drop is larger than the raw board-size ratio (6^2:8^2 = 1:1.78) because the 8×8 network processes a larger input tensor, the action space grows from 37 to 65, and game length increases substantially (more moves per game means more MCTS calls per episode). Peak RAM grows by 26% (95.2 → 120.2 MB), reflecting the larger board representations, extended game histories, and the wider neural network activations. This confirms that moving to 8×8 for production training requires either a larger GPU memory budget or a reduction in active batch size to stay within Colab's 12.7 GB RAM headroom.

7.5.5 Summary of Scalability Findings

Dimension	Finding	Recommended setting
Batch size	Diminishing returns beyond 32 slots; sweet spot is 16–32	**16–32 active slots**
Number of games	Perfectly linear — no efficiency loss at scale	Any; 48 is divisible by 16
MCTS depth	Sims/sec flat across all depths; architecture is depth-robust	35 sims for quality/speed balance
Board size	8×8 costs ~58% more time per game vs. 6×6; plan for it	6×6 for fast iteration, 8×8 for final training

7.6 Phase 3 Benchmark Summary

Path	MCTS sims/sec	Relative speed
`python_tobytes` (baseline path)	1,220.5	1.00
`python_zobrist`	835.4	0.68
`bitboard_cpp`	1,104.2	0.90

The C++ bitboard is faster than the Zobrist implementation (0.90 vs. 0.68) but currently trails the baseline by 10%. The performance gap is entirely attributable to Python–C++ conversion overhead at each call, not to the bitboard logic itself. With board state maintained natively in bitboard form across MCTS traversal (Phase 3b), the C++ path is expected to exceed the Python baseline substantially, particularly given the hardware-POPCNT and AVX2 advantages described in Section 5.7.

---

8. Conclusion

This project demonstrated that the most impactful optimization for AlphaZero-style Othello self-play on Colab T4 is **single-process lockstep batched self-play** (Technique C). By maintaining 16 concurrent active games and coalescing their MCTS leaf requests into shared GPU batches, Technique C raised MCTS throughput from ~1,098 sims/sec to ~1,684 sims/sec — a 53% improvement — while preserving win-rate quality at 1.0.

The project also produced clear negative results that carry their own value:

- **Technique A (NumPy node arrays)**: Array-based vectorization hurts, not helps, for small action spaces (< 100 actions). Measure before optimizing data structures.
- **Technique D (multiprocessing)**: IPC coordination overhead dominates on 2-vCPU hardware regardless of design (two CUDA workers or parent-owned inference). Multi-process MCTS scaling requires many more cores to amortize cross-process latency.
- **Technique E (explicit virtual loss)**: Hot-path bookkeeping regresses throughput even when it correctly preserves quality. Inner-loop additions must be profiled for both call count and per-call cost.

The C++ bitboard back-end (Phase 3) is architecturally sound — AVX2, hardware POPCNT, cache alignment, OpenMP batch functions — but currently undershoots the Python baseline due to boundary-crossing overhead. Eliminating the per-call board reconstruction is the clear next step.

The scalability study (Section 7.5) confirmed three key properties of the Technique C design: throughput scales sub-linearly with batch size (16–32 is the practical sweet spot on T4), total game count scales perfectly linearly with no efficiency loss, and MCTS sims/sec stays flat across a 7.5× range in simulation depth — validating the architecture's inherent depth-robustness. Moving to 8×8 reduces throughput by ~58% versus 6×6, a predictable cost of the larger state space and longer games.

Lessons learned:

1. GPU batch size is the dominant throughput lever when inference is involved; getting from batch size 1 to 11+ was worth more than all other changes combined.
2. The MCTS hot path is extremely sensitive to per-simulation overhead. Even small additions (virtual-loss counters, IPC waits) compound into large regressions at millions of calls.
3. Profiling must be empirical. NumPy vectorization, multiprocessing, and virtual-loss coalescing all appeared theoretically beneficial but measured negatively in this workload.
4. Caching and memory discipline (bounded LRU, array pooling) are always worth doing for robustness, but their throughput contribution is secondary to batch-coordination design.
5. The rolling-refill design is a sound architectural choice: memory and throughput are independent of total game count, making the system well-suited to long training runs without re-tuning.

---

9. LLM Usage Disclosure

In accordance with academic transparency requirements, we disclose the following uses of large-language model assistance in this project.

Where LLMs Were Used

Activity	LLM Role

Activity	LLM Role
Debugging pybind11 compilation errors	Generated fix suggestions for missing include paths and type-cast errors
Designing the `BatchedSelfPlayWorker` coordination loop	Provided initial skeleton; we restructured and extended it
Conversion from python to cpp `othello_bitboard.cpp` creation	Generated code and integration guidelines; we verified correctness and benchmarked
Structuring experiments and metrics	Suggested metric dimensions (batch size, sims/sec, GPU calls); we chose which to track
Drafting this report	Generated full draft based on provided results; we verified all numbers against source JSON/markdown files and edited for accuracy
Debugging the `dotdict` pickle crash in Technique D	Identified `__getstate__`/`__setstate__` fix

Representative Prompts

- *"I have a pybind11 module for Othello. Add OpenMP parallelism and SIMD hints while preserving all existing method signatures."*
- *"Explain why my Technique A implementation regressed performance even though NumPy should be faster than dicts."*
- *"Write a final project report from these result files following this course rubric."*

Limitations and Mitigations

LLM suggestions were treated as starting points, not final answers. In several cases (Technique A regression diagnosis, Technique D failure analysis, Phase 3 benchmark interpretation), the LLM's initial explanation required correction after we ran experiments and compared measured numbers. All quantitative claims in this report were verified against the source JSON metrics files before inclusion. No LLM output was used as a substitute for running experiments.